Project - Phase 2

Mahan Zamani(402101877) - Sina Hashemi(402102668)

---

1 Project Description and Theoretical Questions

1.1 Introduction to License Plate Recognition (LPR)

1.1.1 Two-Dimensional Correlation

1. What is the mathematical definition of correlation between two signals? Explain it in your own words and write the standard formula. What does the correlation value represent?

In this project, we use **Pearson correlation**, which measures how similar the **shape** of two signals is, regardless of their brightness or scale. It tells us whether two patterns rise and fall together (positive correlation), in opposite directions (negative), or have no relation at all (zero).

This type of correlation is **normalized**, meaning it always falls between –1 and +1:

- +1: Perfect match
- 0: No similarity
- –1: Perfect opposite

It is more reliable than raw cross-correlation for comparing image segments, because it eliminates the effect of constant brightness or low contrast.

Formula:: Given two signals $x[n]$ and $y[n]$ of length N:

$$\text{Corr}(x, y) = \frac{\sum_{n=0}^{N-1}(x[n] - \mu_x)(y[n] - \mu_y)}{\sqrt{\sum(x[n] - \mu_x)^2} \cdot \sqrt{\sum(y[n] - \mu_y)^2}}$$

Where:

- μ_x: mean of signal x
- μ_y: mean of signal y
- The result lies in the range $[-1, 1]$

This method works well for **pattern matching**, even if the overall intensity (brightness) of the image changes, which is useful in real-world conditions.

---

2. Research and explain how a method designed for one-dimensional signals can be used to compare images or image segments. Describe your proposed approach clearly. (Do not explain 2D correlation.)

Although images are 2D, we can apply 1D Pearson correlation in two passes — first across rows, then across columns — to compute similarity without using 2D correlation.

Here's a step-by-step method:

1. Ensure the **template image** (e.g., character 'A') and the **target image** (license plate character segment) are the same size: $N \times M$.
2. Normalize both to grayscale and resize/crop if needed.
3. For each row $i \in [0, N-1]$:

- Extract row vectors: $x_i = \text{template}[i, :]$ $y_i = \text{target}[i, :]$
- Compute **Pearson correlation** between x_i and y_i.
- Store the result as $r_{\text{row}}[i]$

4. Compute the **average of all row-wise correlation values**:

$$\text{score}_{\text{row}} = \frac{1}{N} \sum_{i=0}^{N-1} r_{\text{row}}[i]$$

5. For each column $j \in [0, M-1]$:

- Extract column vectors: $x_j = \text{template}[:, j]$ $y_j = \text{target}[:, j]$
- Compute **Pearson correlation** between x_j and y_j.
- Store the result as $r_{\text{col}}[j]$

6. Compute the **average of all column-wise correlation values**:

$$\text{score}_{\text{col}} = \frac{1}{M} \sum_{j=0}^{M-1} r_{\text{col}}[j]$$

7. Combine both row and column scores using a weighted average:

$$\text{final score} = \frac{\text{score}_{\text{row}} + \text{score}_{\text{col}}}{2}$$

This method compares both **horizontal and vertical patterns** using only 1D correlation. It is more sensitive to noise, rotation, and misalignment than full 2D correlation, but it's computationally simpler.

1.1.2 Image Blurring and Degradation Modeling

1. Explain how this 1D filter is applied to images, and also explain the parameter p and its effect on the filter's performance.

$$H_{blur} = \frac{1-p}{1-pz^{-1}}$$

Filter Properties:

- Type: Infinite Impulse Response (IIR)
- Frequency Response: Low-pass
- zeros & poles: one zero at 0 one pole at p

Impact on the Spatial Domain:

The filter operates using the following recurrence relation:

$$y[n] = (1-p)x[n] + py[n-1]$$

Observed Effect on the Image:

The filter introduces **motion blur**, which aligns with theoretical expectations:

- Low-pass filters blur images by attenuating high-frequency details
- The equation shows each pixel's intensity depends on its previous value, mimicking motion blur's temporal propagation

Impact of Parameter p:

When analyzing the pole-zero plot:

- The filter has a single pole on the unit circle
- As p increases:
  - The pole moves closer to the unit circle
  - Low-pass effect intensifies
  - Image blurring becomes stronger

Filter Application on Images:

Since the filter is one-dimensional (1D), it cannot be directly applied to a 2D image. However, it can be applied either row-wise or column-wise.

---

2. If we assume that we have the blur transformation function (blur kernel) of the channel, can we recover a high-quality and clean image? Why?

The Problem: Naive Deconvolution Fails

Given a blurry image y, blur kernel h, and noise n, the true image x satisfies:

$$y = h * x + n$$

where * denotes convolution.

The naive solution

$$X = Y/H$$

(in Fourier domain) fails because:

1. Zeros in H : Division by zero causes instability.
2. Noise Amplification: N / H corrupts the estimate.

Wiener Deconvolution: Theory

The Wiener filter minimizes the Mean Squared Error (MSE) between X and its estimate X'.

Frequency-Domain Formula

$$\hat{X} = \left[\frac{H^*}{|H|^2 + NSR}\right] Y$$

- H*: Complex conjugate of blur kernel H.
- NSR: Noise-to-signal ratio.

1.1.3 Sampling Rate and Resolution Effects

1. Do a bit of research on 2D sampling theory and briefly summarize your understanding.

2D sampling theory extends the principles of Nyquist-Shannon sampling theory from 1D signals (e.g., audio) to 2D signals (e.g., images). The key idea is that a continuous 2D signal (like an optical image) must be sampled at a sufficient rate to avoid aliasing (distortion caused by undersampling).

---

2 Implementation

In [1]:
```
import os
import cv2
import numpy as np
from scipy import fftpack
import matplotlib.pyplot as plt
```

2.1 Ideal Case – Controlled Environment

2.1.1. Write a function that takes a license plate image as input and split the image into its individual characters (both digits and letters). Plot each segmented character.

In [2]:

```python
def segment_characters(plate_image_path):
    """
    Segments characters from a license plate image.

    Parameters:
    plate_image_path (str/np.array): File path OR preloaded grayscale ima

    Returns:
    list: Grayscale character images (80x40 numpy arrays)
    """

    # Step 1: Load and convert to grayscale
    try:
        image = cv2.imread(plate_image_path)
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    except Exception as e:
        # To support the second part of the program
        image = plate_image_path
        gray = image

    # Step 2: Threshold the image to binary
    # characters(dark) -> white, background(light) -> black
    mean_val = np.mean(gray)
    binary = np.where(gray > mean_val, 0, 255).astype(np.uint8)

    # Step 3: Remove borders and blue area on the left
    # We'll skip the first 15% of columns to avoid EU symbol and blue str
    binary[:, :int(0.15 * binary.shape[1])] = 0

    # Step 4: Use contours to find character regions
    contours, _ = cv2.findContours(binary, cv2.RETR_EXTERNAL, cv2.CHAIN_A

    # Step 5: Filter and sort bounding boxes (only keep likely characters
    char_boxes = []
    for cnt in contours:
        x, y, w, h = cv2.boundingRect(cnt)
        aspect_ratio = h / w
        if 0.5 < aspect_ratio < 5 and h > 0.4 * binary.shape[0]:  # likel
            char_boxes.append((x, y, w, h))

    # Sort left to right
    char_boxes = sorted(char_boxes, key=lambda b: b[0])

    # Step 6: Extract character images
    characters = []
    for x, y, w, h in char_boxes:
        char_img = gray[y-4:y+h+4, x-4:x+w+4]
        char_img = cv2.resize(char_img, (40, 80))  # Ensure size matches
        characters.append(char_img)

    return characters
```

The `segment_characters` function uses `cv2.imread` to load the image from a file path and `cv2.cvtColor(..., cv2.COLOR_BGR2GRAY)` to convert the image from BGR (color) to grayscale.

Then, it calculates the mean pixel value of the grayscale image. The function converts bright areas (assumed to be background) to black (0), and darker regions

(characters) to white (255). This manual thresholding method is useful when adaptive methods fail due to uniform lighting or embossing.

After removing leftmost 15% of the image, it finds contours of character regions.

- `cv2.findContours` : Detects contours (closed shapes) in the binary image.
- `cv2.RETR_EXTERNAL` : Only retrieves external contours (ignores nested ones).
- `cv2.CHAIN_APPROX_SIMPLE` : Compresses horizontal, vertical, and diagonal segments to save memory.

After filtering out contours that are too small or have unrealistic shape (like noise), Characters are then sorted from left to right using the `x` coordinate.

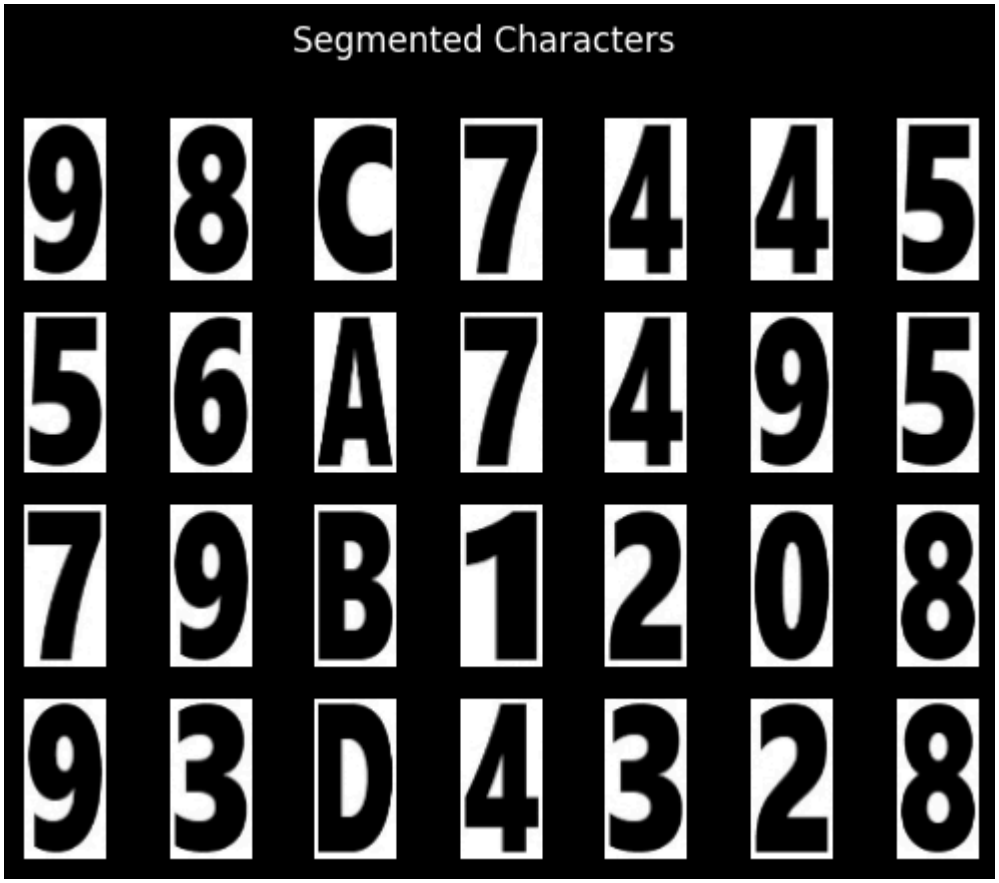
Each character is cropped from the original grayscale image with a 4-pixel padding to preserve borders. Then resized to 40×80 pixels to match the reference template size.

In [3]:
```
idealCharsList = []

for i in range(4):
    idealCharsList.append(segment_characters(f"../ideal/p{i+1}.jpg"))

fig1, ax = plt.subplots(4, 7)
for i in range(4):
        for j in range(len(idealCharsList[i])):
            ax[i][j].imshow(idealCharsList[i][j], cmap='gray')
            ax[i][j].axis('off')
fig1.suptitle('Segmented Characters')
```

Out[3]:
```
Text(0.5, 0.98, 'Segmented Characters')
```



2.1.2. Now, using the provided technique, recognize each character and print the final license plate as output. Make sure to report the correlation value obtained for character recognition.(Do this for all the license plates in "ideal" folder.)

In [4]:

```
def pearson_corr(x, y):
    """
    Computes an averaged Pearson correlation between two 2D arrays (image
    by applying 1D correlation row-wise and column-wise.

    Parameters:
    x (np.array): First grayscale image
    y (np.array): Second grayscale image

    Returns:
    float: Averaged correlation value
    """
    x = x.astype(np.float32)
    y = y.astype(np.float32)

    row_corrs = []
    for i in range(x.shape[0]):  # for each row
        r1 = x[i, :]
        r2 = y[i, :]
        if np.std(r1) == 0 or np.std(r2) == 0:
            corr = 0
        else:
            corr = np.corrcoef(r1, r2)[0, 1]
        row_corrs.append(corr)

    col_corrs = []
    for j in range(x.shape[1]):  # for each column
        c1 = x[:, j]
        c2 = y[:, j]
        if np.std(c1) == 0 or np.std(c2) == 0:
            corr = 0
        else:
            corr = np.corrcoef(c1, c2)[0, 1]
        col_corrs.append(corr)

    # Combine row and column correlations
    all_corrs = row_corrs + col_corrs
    return np.mean(all_corrs)


def load_templates(folder):
    """
    Loads all character templates from a folder and resizes them.

    Parameters:
    folder (str): Path to a folder containing character images

    Returns:
    dict: Dictionary mapping character labels (str) to grayscale images (
    """
    templates = {}

    for filename in os.listdir(folder):
        if filename.endswith(".jpg"):
```

```
            label = os.path.splitext(filename)[0]  # Extract character la
            img = cv2.imread(os.path.join(folder, filename), cv2.IMREAD_G
            img = cv2.resize(img, (40, 80))  # Normalize size
            templates[label] = img

    return templates

def recognize_plate(char_images, letters_path="../letters", numbers_path=
    """
    Recognizes a sequence of characters from a list of segmented characte

    Parameters:
    char_images (list): List of 40x80 grayscale images of individual char
    letters_path (str): Path to folder containing letter templates
    numbers_path (str): Path to folder containing digit templates

    Returns:
    tuple:
        - recognized (str): Recognized license plate string
        - correlations (list of dict): Correlation values per character p
        - bestCorrelations (list of float): Highest correlation for each
    """
    # Load all templates
    templates = dict()
    templates.update(load_templates(letters_path))
    templates.update(load_templates(numbers_path))

    recognized = ""
    correlations = []      # Full dictionary of correlation values for ea
    bestCorrelations = []  # Only best values for reporting

    for i, char_img in enumerate(char_images):
        charCorr = dict()
        char_img = cv2.resize(char_img, (40, 80))  # Ensure the input cha

        best_char = '?'
        best_corr = -1

        # Compare each character with every template using Pearson correl
        for label, tmpl_img in templates.items():
            corr = pearson_corr(char_img, tmpl_img)
            charCorr[label] = corr

            if corr > best_corr:
                best_corr = corr
                best_char = label

        recognized += best_char
        correlations.append(charCorr)
        bestCorrelations.append(best_corr)

    return recognized, correlations, bestCorrelations
```

In [5]:

```
for i in range(4):
    recognized, corr_vals, best_corrs = recognize_plate(idealCharsList[i]
    print(recognized)
    print("Best Correlations: ", best_corrs)
    print("All Correlations:")
```

```
    for corr_val in corr_vals:
        print(corr_val)
```

98C7445
Best Correlations: [0.9061192299115105, 0.9043874793026419, 0.9157060409047278, 0.9157735057407074, 0.9414255362599449, 0.9227800565784415, 0.8995899987246314]
All Correlations:
{'B': 0.5421406014526868, 'C': 0.16416126491083716, 'A': 0.222938743410532, 'D': 0.5909924028294132, '8': 0.576040705617693, '9': 0.9061192299115105, '4': 0.19785129327667506, '5': 0.5735391447415542, '7': 0.25519829493572865, '6': 0.491149451223296, '2': 0.33323861835011037, '3': 0.5645932807796361, '1': 0.36356316377895354, '0': 0.6714878034069919}
{'B': 0.727832846987157, 'C': 0.32425311704079307, 'A': 0.2697889858988129, 'D': 0.41560088854110594, '8': 0.9043874793026419, '9': 0.5450588368144623, '4': 0.4335046897330983, '5': 0.5128552585285315, '7': 0.2186560852526645, '6': 0.5914640847235446, '2': 0.4365693635174278, '3': 0.5841595823118194, '1': 0.3186995670937262, '0': 0.5973993054823715}
{'B': 0.246470825199943, 'C': 0.9157060409047278, 'A': 0.2125875443066817, 'D': 0.1644178020185253, '8': 0.32092276762160643, '9': 0.15976787332823583, '4': 0.335016554619694, '5': 0.18651872617898058, '7': 0.025197211295719908, '6': 0.5119755904551061, '2': 0.1737791309665533, '3': 0.010718840867625463, '1': 0.13626483471648326, '0': 0.4403676809210643}
{'B': 0.25557627859242, 'C': 0.01740530673222658, 'A': 0.09088022685197135, 'D': 0.1444134530841281, '8': 0.22539651219364976, '9': 0.2448733942555083, '4': 0.14379483828776388, '5': 0.21108440328401581, '7': 0.9157735057407074, '6': 0.08682198950188286, '2': 0.4314291125350762, '3': 0.38714258704809457, '1': 0.29581594839406644, '0': 0.10867932919229141}
{'B': 0.3339043622043484, 'C': 0.3767382945429733, 'A': 0.47367070899202113, 'D': 0.19753523531903744, '8': 0.4425211051156722, '9': 0.20504940298506288, '4': 0.9414255362599449, '5': 0.28417717654817304, '7': 0.12981933922855118, '6': 0.43273306528203126, '2': 0.2691243146444988, '3': 0.35722715618996986, '1': 0.32964318902495077, '0': 0.3622352876150013}
{'B': 0.3254984791468676, 'C': 0.37389456059538184, 'A': 0.476549622861835, 'D': 0.1952468412401092, '8': 0.4369754507554313, '9': 0.19417804566851074, '4': 0.9227800565784415, '5': 0.28814463313289923, '7': 0.1287793374768723, '6': 0.4323732866832657, '2': 0.2686463002717394, '3': 0.35610633681901643, '1': 0.33101953342468154, '0': 0.3587848021165659}
{'B': 0.5030076178694105, 'C': 0.19038054036136653, 'A': 0.22079649594933323, 'D': 0.35186307317263943, '8': 0.4958075039585629, '9': 0.537079295903321, '4': 0.2658833018567932, '5': 0.8995899987246314, '7': 0.20727479319080658, '6': 0.5821975094131898, '2': 0.19122210221997715, '3': 0.5742223741192352, '1': 0.3312669141616593, '0': 0.3940834737269749}
56A7495
Best Correlations: [0.9117335334662097, 0.914380091064046, 0.9310668161813303, 0.9072011731228427, 0.9338652378863764, 0.8967714959011837, 0.8994102607264911]
All Correlations:
{'B': 0.5007847574467325, 'C': 0.19529285361670393, 'A': 0.21790179907491414, 'D': 0.3533395779432566, '8': 0.5021564561677575, '9': 0.5365848109778713, '4': 0.27396061026229457, '5': 0.9117335334662097, '7': 0.2089769149428846, '6': 0.5829136984561293, '2': 0.1895102300194207, '3': 0.5845057623511529, '1': 0.335069039536841, '0': 0.39242427940106545}
{'B': 0.552410280963691, 'C': 0.5090529634323169, 'A': 0.2632954560964162, 'D': 0.4079837631541328, '8': 0.6314013052760458, '9': 0.4899401868115334, '4': 0.41780410676462915, '5': 0.5908183232340193, '7': 0.0926564253324587, '6': 0.914380091064046, '2': 0.21892112935023914, '3': 0.34131024194765386, '1': 0.2795007024377757, '0': 0.6791389103076763}
{'B': 0.2728166955362716, 'C': 0.2274130245388654, 'A': 0.931066816181303, 'D': 0.18891125366939918, '8': 0.2985163692703611, '9': 0.2515221571400956, '4': 0.48808259650622676, '5': 0.2394520574413539, '7': 0.08595049376248293, '6': 0.2842468469905736, '2': 0.34920048906405504, '3': 0.30065887287844695, '1': 0.1791243535038275, '0': 0.2683715950820552}

{'B': 0.2525973319780484, 'C': 0.024033458012113165, 'A': 0.09203595744533
312, 'D': 0.14500607109441938, '8': 0.22717108148676451, '9': 0.2472301773
7685418, '4': 0.15004902707133808, '5': 0.21547803600742402, '7': 0.907201
1731228427, '6': 0.09337917400379322, '2': 0.4386312158083454, '3': 0.3908
7010561271146, '1': 0.2863514370793024, '0': 0.10716036230706798}
{'B': 0.3367183956683152, 'C': 0.3785449961608158, 'A': 0.473518225612095
6, 'D': 0.19788000461567817, '8': 0.44573303057511887, '9': 0.208008773368
7429, '4': 0.9338652378863764, '5': 0.28636963553028416, '7': 0.1306980561
9447541, '6': 0.440522363007157, '2': 0.26817456640460957, '3': 0.35801914
426998227, '1': 0.3307971834453134, '0': 0.3741834002841253}
{'B': 0.5396632670737534, 'C': 0.16924390388774918, 'A': 0.220927406128418
5, 'D': 0.5938354244580963, '8': 0.5702493236695934, '9': 0.89677149590118
37, '4': 0.2056123172137188, '5': 0.5732427672127488, '7': 0.2462719740472
4108, '6': 0.4915464135425507, '2': 0.3441792798173546, '3': 0.57168644630
52034, '1': 0.365195226463752, '0': 0.6619544835066}
{'B': 0.4988743753858395, 'C': 0.18738768748677767, 'A': 0.22746129003213
5, 'D': 0.3459168615033568, '8': 0.5020475917049552, '9': 0.53880460787943
36, '4': 0.27528684226297295, '5': 0.8994102607264911, '7': 0.195632926027
02836, '6': 0.5833783354211175, '2': 0.18745841609004923, '3': 0.579469974
9615982, '1': 0.3323006297283115, '0': 0.39293803446057657}
79B1208
Best Correlations: [0.9260999340738685, 0.9102742846727494, 0.895656724254
48273, 0.9097511479878314, 0.9088175935702167, 0.9081527589580239, 0.91525
27795360859]
All Correlations:
{'B': 0.24757022132258666, 'C': 0.025126738617640944, 'A': 0.0865307915484
0661, 'D': 0.14617028381149527, '8': 0.22954529956404338, '9': 0.252143479
1225325, '4': 0.1385684022763904, '5': 0.21350978042716917, '7': 0.9260999
340738685, '6': 0.09138223225042345, '2': 0.439219260497753, '3': 0.395611
15622818654, '1': 0.2906943378810947, '0': 0.11079892246760495}
{'B': 0.5465059360929793, 'C': 0.16186718262578015, 'A': 0.223491683471183
37, 'D': 0.5896212504440075, '8': 0.570609252702249, '9': 0.91027428467274
94, '4': 0.19912861070273818, '5': 0.5705626421819378, '7': 0.256220765283
658, '6': 0.48979175355307103, '2': 0.3428363549907128, '3': 0.56571628174
7532, '1': 0.3689105542209783, '0': 0.6670315101736134}
{'B': 0.8956567242548273, 'C': 0.2539213857362473, 'A': 0.2273988538326379
4, 'D': 0.5780217096347757, '8': 0.6703123219752481, '9': 0.47663709436654
783, '4': 0.3014121438771163, '5': 0.46435007314958104, '7': 0.24251985220
485733, '6': 0.4980512311847388, '2': 0.374179934026316, '3': 0.5419180989
996344, '1': 0.22212885774255134, '0': 0.5063809308660842}
{'B': 0.21963663546530376, 'C': 0.16203445147444653, 'A': 0.16653865957833
02, 'D': 0.17456677071577176, '8': 0.2914371197887269, '9': 0.357410732639
15285, '4': 0.30672127966216106, '5': 0.3054851976460241, '7': 0.313728015
31493016, '6': 0.25678072804497776, '2': 0.2986084440450171, '3': 0.325910
01498021027, '1': 0.9097511479878314, '0': 0.25197692308303576}
{'B': 0.3928790779189009, 'C': 0.16941993384817614, 'A': 0.32729555048481
5, 'D': 0.2632809715161033, '8': 0.4125797776837286, '9': 0.31251265323336
01, '4': 0.2462949847297352, '5': 0.17521451620277123, '7': 0.462042444012
04886, '6': 0.19290224615361373, '2': 0.9088175935702167, '3': 0.437523342
9222433, '1': 0.3019800341921631, '0': 0.29927384547459646}
{'B': 0.5693038718430955, 'C': 0.4237105888975633, 'A': 0.2224789562699331
6, 'D': 0.6605357991606716, '8': 0.6465442682464643, '9': 0.64089943023193
81, '4': 0.3450897676077477, '5': 0.4045748047789967, '7': 0.1028342848337
9259, '6': 0.6522449009011065, '2': 0.31422971946963935, '3': 0.3779094193
7217504, '1': 0.258913874371002, '0': 0.9081527589580239}
{'B': 0.7278079228863853, 'C': 0.32451064763734117, 'A': 0.264537469732046
34, 'D': 0.41741015131435794, '8': 0.9152527795360859, '9': 0.546330043032
1412, '4': 0.43288207931839773, '5': 0.5113418435555052, '7': 0.2247703083
751326, '6': 0.5908992115709819, '2': 0.4406932646523407, '3': 0.584545639
2555171, '1': 0.31758898883748876, '0': 0.5912863597185013}

```
93D4328
Best Correlations:  [0.8926488325806613, 0.918758252892535, 0.912998267774
7566, 0.9361323379202227, 0.8881080088417622, 0.8785847549042836, 0.898148
6286180308]
All Correlations:
{'B': 0.5358910059037528, 'C': 0.17387907451608597, 'A': 0.215368724470577
6, 'D': 0.5866700084068055, '8': 0.5696360761490451, '9': 0.89264883258066
13, '4': 0.2050084664441741, '5': 0.5750545420615256, '7': 0.2501696100986
058, '6': 0.495958696369798, '2': 0.3429791766615104, '3': 0.5720468894570
369, '1': 0.3737861240474764, '0': 0.657694228511644}
{'B': 0.5874959550676953, 'C': 0.026758461008790778, 'A': 0.30056074084852
54, 'D': 0.3865938784435643, '8': 0.5781107677988507, '9': 0.5237531573763
98, '4': 0.37012754416094373, '5': 0.5626800642058414, '7': 0.395261559413
44293, '6': 0.3185429036726141, '2': 0.43466114971660874, '3': 0.918758252
892535, '1': 0.34844348351273285, '0': 0.34672986007605405}
{'B': 0.5844840351021522, 'C': 0.15556042620659863, 'A': 0.144061751246630
98, 'D': 0.9129982677747566, '8': 0.4114651748427917, '9': 0.5478163664713
823, '4': 0.167353121972129, '5': 0.3548401254082561, '7': 0.1300998116958
7106, '6': 0.38705993960294394, '2': 0.26682560706345293, '3': 0.386207725
6612589, '1': 0.16297164160552227, '0': 0.6472151771690892}
{'B': 0.32868054024108595, 'C': 0.3737139399988123, 'A': 0.479252222069253
8, 'D': 0.19478091591696198, '8': 0.44071123630818565, '9': 0.203322899059
61345, '4': 0.9361323379202227, '5': 0.2853449612406941, '7': 0.1251929607
7554004, '6': 0.4335029055589036, '2': 0.2721963622981545, '3': 0.36123203
46694235, '1': 0.32301369421897885, '0': 0.3571487257896332}
{'B': 0.6028058037060383, 'C': 0.02304022239744707, 'A': 0.282021465568444
3, 'D': 0.3843163891267138, '8': 0.5821673339956607, '9': 0.51584653101304
27, '4': 0.3635143172652883, '5': 0.5616043292839454, '7': 0.3894252369946
8057, '6': 0.3131378546031421, '2': 0.43935552430400254, '3': 0.8881080088
417622, '1': 0.350902068526903, '0': 0.34252223957750044}
{'B': 0.3913403190305554, 'C': 0.1708771592645656, 'A': 0.3199004638869099
6, 'D': 0.2582941806390563, '8': 0.41917774268394103, '9': 0.3134901351963
748, '4': 0.2525052986671182, '5': 0.19039671288741547, '7': 0.46274472740
12942, '6': 0.1988072566297216, '2': 0.8785847549042836, '3': 0.4487531850
783147, '1': 0.2960177697407104, '0': 0.29712027729877366}
{'B': 0.727414011291513, 'C': 0.32098459794018164, 'A': 0.263493896289967
3, 'D': 0.4200990116266099 5, '8': 0.8981486286180308, '9': 0.5411277668524
27, '4': 0.42206342191841034, '5': 0.5201967268868019, '7': 0.218729176002
72995, '6': 0.586966848993454, '2': 0.44303330390143714, '3': 0.5828268599
64684, '1': 0.32124462154906036, '0': 0.5916049648381633}
```

2.1.3. In this step, analyze the effect of spatial downsampling (i.e., resolution reduction) on character recognition accuracy.

- Apply downsampling to the license plate image before performing character segmentation and also plot before and after.
- Repeat the character segmentation and recognition process for multiple downsampling rates (e.g., 1/2, 1/4, 1/6, ...).
- For each rate, evaluate whether the characters are still recognizable using your correlation-based method.
- Determine and report the minimum sampling rate at which the license plate is still correctly recognizable.
- Plot or tabulate your findings to show how recognition accuracy changes with sampling rate.
- It is not necessary to include all the processes for all the license plates in "ideal" folder in the report. Therefore, include the processing steps for one license plate

image in the report, and only report the final outputs for the remaining license plates .

In [6]:
```
def downsample(img, factor):
    """
    Downsample by selecting every 'factor'-th pixel.
    """
    downsampled = img[::factor, ::factor]
    return downsampled
```

In [7]:
```
idealPlate1 = cv2.imread('../ideal/p1.jpg', cv2.IMREAD_GRAYSCALE)
idealPlate2 = cv2.imread('../ideal/p2.jpg', cv2.IMREAD_GRAYSCALE)
idealPlate3 = cv2.imread('../ideal/p3.jpg', cv2.IMREAD_GRAYSCALE)
idealPlate4 = cv2.imread('../ideal/p4.jpg', cv2.IMREAD_GRAYSCALE)
idealPlates = [idealPlate1, idealPlate2, idealPlate3, idealPlate4]

for j in range(4):
    plt.figure(figsize=(12, 8))
    for i in range(5):
        temp = downsample(idealPlates[j], 2**i)
        recognized, corr_vals, best_corrs = recognize_plate(segment_chara
        print(f"for plate {j+1}, Down-Sampling 1/{2**i} We have recognize
        plt.subplot(1, 5, i+1), plt.imshow(temp, cmap='gray'), plt.title(
    plt.tight_layout()
    plt.show()
```

```
for plate 1, Down-Sampling 1/1 We have recognized plate: 98C7445
for plate 1, Down-Sampling 1/2 We have recognized plate: 98C7445
for plate 1, Down-Sampling 1/4 We have recognized plate: 98C7445
for plate 1, Down-Sampling 1/8 We have recognized plate: AAA7A4
for plate 1, Down-Sampling 1/16 We have recognized plate: BBBBBB
```




```
for plate 2, Down-Sampling 1/1 We have recognized plate: 56A7495
for plate 2, Down-Sampling 1/2 We have recognized plate: 56A7495
for plate 2, Down-Sampling 1/4 We have recognized plate: 54A7495
for plate 2, Down-Sampling 1/8 We have recognized plate: 44A7144
for plate 2, Down-Sampling 1/16 We have recognized plate: BBBBBB
```




```
for plate 3, Down-Sampling 1/1 We have recognized plate: 79B1208
for plate 3, Down-Sampling 1/2 We have recognized plate: 79B1208
for plate 3, Down-Sampling 1/4 We have recognized plate: 7981208
for plate 3, Down-Sampling 1/8 We have recognized plate: 7AA4744
for plate 3, Down-Sampling 1/16 We have recognized plate: BBBBB
```




```
for plate 4, Down-Sampling 1/1 We have recognized plate: 93D4328
for plate 4, Down-Sampling 1/2 We have recognized plate: 93D4328
for plate 4, Down-Sampling 1/4 We have recognized plate: 9304328
for plate 4, Down-Sampling 1/8 We have recognized plate: A7A447A
for plate 4, Down-Sampling 1/16 We have recognized plate: BBBBB
```



Due to experiments result minimum sampling rate which license plates are still recognizable is 1/4.

2.2 Realistic Case – Simulated Real-World Conditions

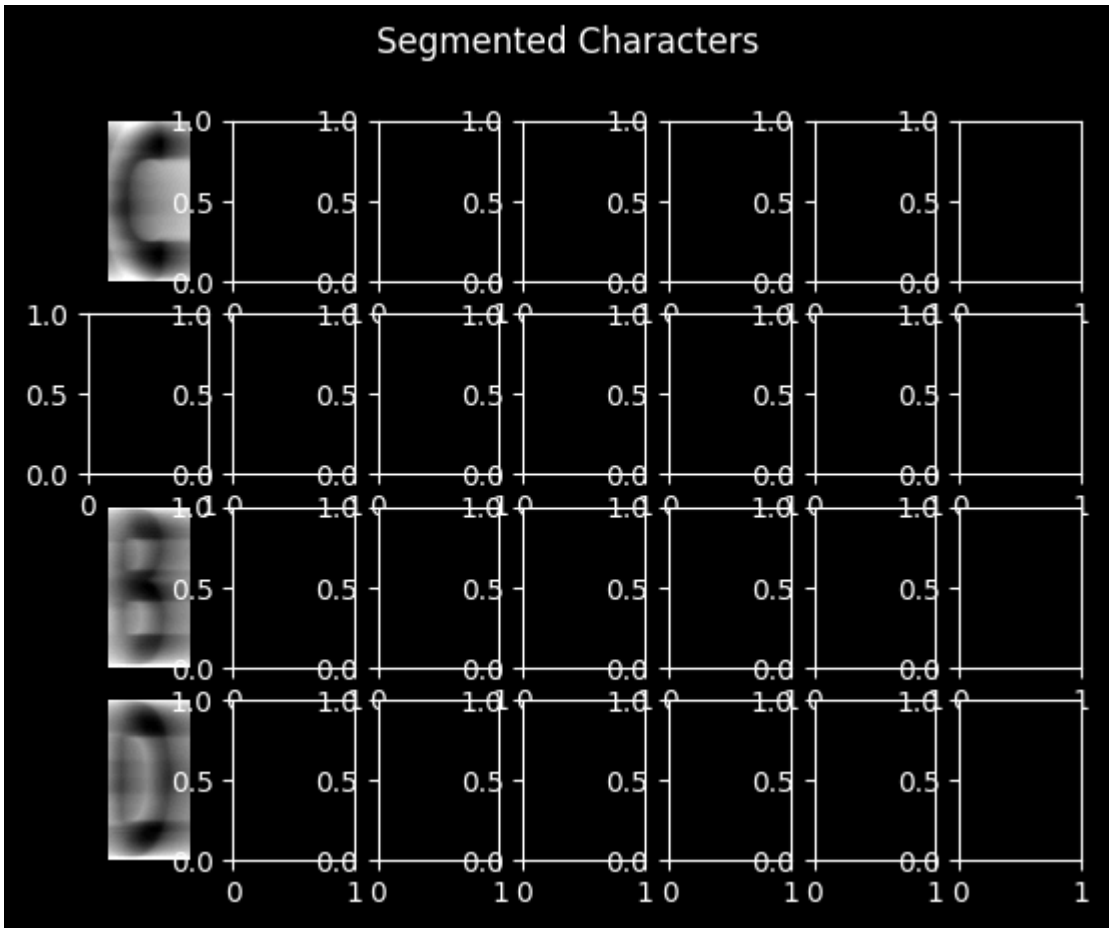
2.2.1. Attempt to extract the license plate characters using the functions from the previous section. Evaluate whether character recognition is successful without any deblurring.

In [8]:
```
realisticCharsList = []

for i in range(4):
    realisticCharsList.append(segment_characters(f"../realistic/p{i+1}.jp

fig2, ax = plt.subplots(4, 7)
for i in range(4):
        for j in range(len(realisticCharsList[i])):
            ax[i][j].imshow(realisticCharsList[i][j], cmap='gray')
            ax[i][j].axis('off')
fig2.suptitle('Segmented Characters')
```

Out[8]:
```
Text(0.5, 0.98, 'Segmented Characters')
```



In [9]:
```
for i in range(4):
    recognized, corr_vals, best_corrs = recognize_plate(realisticCharsLis
    print(recognized)
    print("Best Correlations: ", best_corrs)
```

```
C
Best Correlations:  [0.763668432066349]

Best Correlations:  []
3
Best Correlations:  [0.6216059953835085]
0
Best Correlations:  [0.649577287890637]
```

As is clear from the results, our function is not capable of reading a real license plate.

2.2.2. Since your answer to the previous section is likely negative, you should now use the available information to extract higher-quality and higher-resolution images. Estimate the optimal value of p and use it to find deblur filter.

As explained in Section 1.1.2, p represents the intensity of the filter. A higher value of p (where $0 < p < 1$) results in a more blurred image during filtering, while during deblurring, a higher p value leads to a higher-quality recovered image.

Based on experimental results, $p = 0.99$ has been found to be an optimal value for achieving high-quality image recovery. However, it is important to note that setting p too close to 1 (or exceeding reasonable limits) can introduce excessive distortion, negatively affecting the output. Therefore, values should be carefully chosen to balance clarity and artifacts.

2.2.3. The blur in the provided license plate images is modeled using a one-dimensional linear filter. As discussed earlier, such filters can be applied to two-dimensional images in horizontal and vertical directions. Apply the filter you obtained in the previous section (with the estimated optimal parameter p) to deblur the image. Plot and save the deblurred images. Select the clearest version for further processing.

In [10]:
```python
def recover_motion_blurry_img(img, p=0.99, direction='right'):
    """
    Recover a motion-blurred image with specified direction.

    Parameters:
    - img: Input blurred image (grayscale)
    - p: Blur parameter (0 < p < 1)
    - direction: Blur direction ('left', 'right', 'up', 'down')
    """
    img = img.astype(np.float32) / 255.0  # Convert to float32 and normal

    org_M, org_N = img.shape

    # Determine padding based on direction
    if direction in ('left', 'right'):
        c = org_N
        paddedImg = cv2.copyMakeBorder(img, 0, 0, c, c, cv2.BORDER_REPLIC
    else:  # up or down
        c = org_M
        paddedImg = cv2.copyMakeBorder(img, c, c, 0, 0, cv2.BORDER_REPLIC

    M, N = paddedImg.shape
```

```
    # Create the appropriate filter based on direction
    if direction in ('left', 'right'):
        u = np.arange(N)
        omega = 2 * np.pi * u / N
        if direction == 'right':
            H = (1 - p) / (1 - p * np.exp(-1j * omega))
        else:  # left
            H = (1 - p) / (1 - p * np.exp(1j * omega))
        H = np.tile(H, (M, 1))
    else:  # up or down
        v = np.arange(M)
        omega = 2 * np.pi * v / M
        if direction == 'down':
            H = (1 - p) / (1 - p * np.exp(-1j * omega))
        else:  # up
            H = (1 - p) / (1 - p * np.exp(1j * omega))
        H = np.tile(H, (N, 1)).T  # Transpose for column-wise operation

    # Frequency domain processing
    G = fftpack.fft2(paddedImg)
    F_est = G / H
    deblurred_image = np.real(fftpack.ifft2(F_est))

    # Crop to original size
    if direction in ('left', 'right'):
        deblurred_image = deblurred_image[:, c:c+org_N]
    else:
        deblurred_image = deblurred_image[c:c+org_M, :]

    # Gaussian Filter for cleaner images
    deblurred_image = cv2.GaussianBlur(deblurred_image, (5, 5), 0)

    return deblurred_image

# Example usage
realImg1 = cv2.imread('../realistic/p1.jpg', cv2.IMREAD_GRAYSCALE)
realImg2 = cv2.imread('../realistic/p2.jpg', cv2.IMREAD_GRAYSCALE)
realImg3 = cv2.imread('../realistic/p3.jpg', cv2.IMREAD_GRAYSCALE)
realImg4 = cv2.imread('../realistic/p4.jpg', cv2.IMREAD_GRAYSCALE)

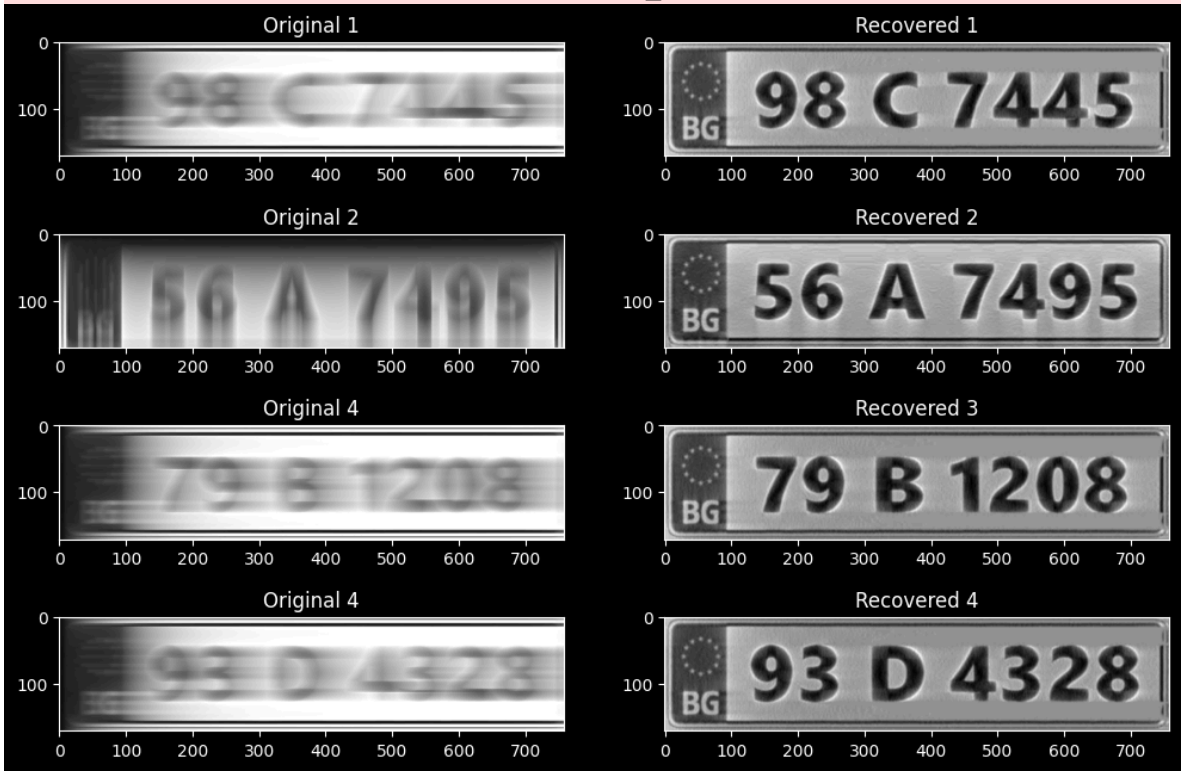
# Process with different directions
recRealImg1 = recover_motion_blurry_img(realImg1, 0.99, 'right')
recRealImg2 = recover_motion_blurry_img(realImg2, 0.99, 'down')
recRealImg3 = recover_motion_blurry_img(realImg3, 0.99, 'right')
recRealImg4 = recover_motion_blurry_img(realImg4, 0.99, 'right')


try:
    os.mkdir("../outputs/")
except Exception as e:
    pass
cv2.imwrite("../outputs/recRealImg1.jpg", cv2.normalize(recRealImg1, None
cv2.imwrite("../outputs/recRealImg2.jpg", cv2.normalize(recRealImg2, None
cv2.imwrite("../outputs/recRealImg3.jpg", cv2.normalize(recRealImg3, None
cv2.imwrite("../outputs/recRealImg4.jpg", cv2.normalize(recRealImg4, None


plt.figure(figsize=(12, 8))
plt.subplot(4, 2, 1), plt.imshow(realImg1, cmap='gray'), plt.title('Origi
plt.subplot(4, 2, 2), plt.imshow(recRealImg1, cmap='gray'), plt.title('Re
```

```
plt.subplot(4, 2, 3), plt.imshow(realImg2, cmap='gray'), plt.title('Origi
plt.subplot(4, 2, 4), plt.imshow(recRealImg2, cmap='gray'), plt.title('Re
plt.subplot(4, 2, 5), plt.imshow(realImg3, cmap='gray'), plt.title('Origi
plt.subplot(4, 2, 6), plt.imshow(recRealImg3, cmap='gray'), plt.title('Re
plt.subplot(4, 2, 7), plt.imshow(realImg4, cmap='gray'), plt.title('Origi
plt.subplot(4, 2, 8), plt.imshow(recRealImg4, cmap='gray'), plt.title('Re
plt.show()
```

```
[ WARN:0@153.763] global loadsave.cpp:848 imwrite_ Unsupported depth image
for selected encoder is fallbacked to CV_8U.
```



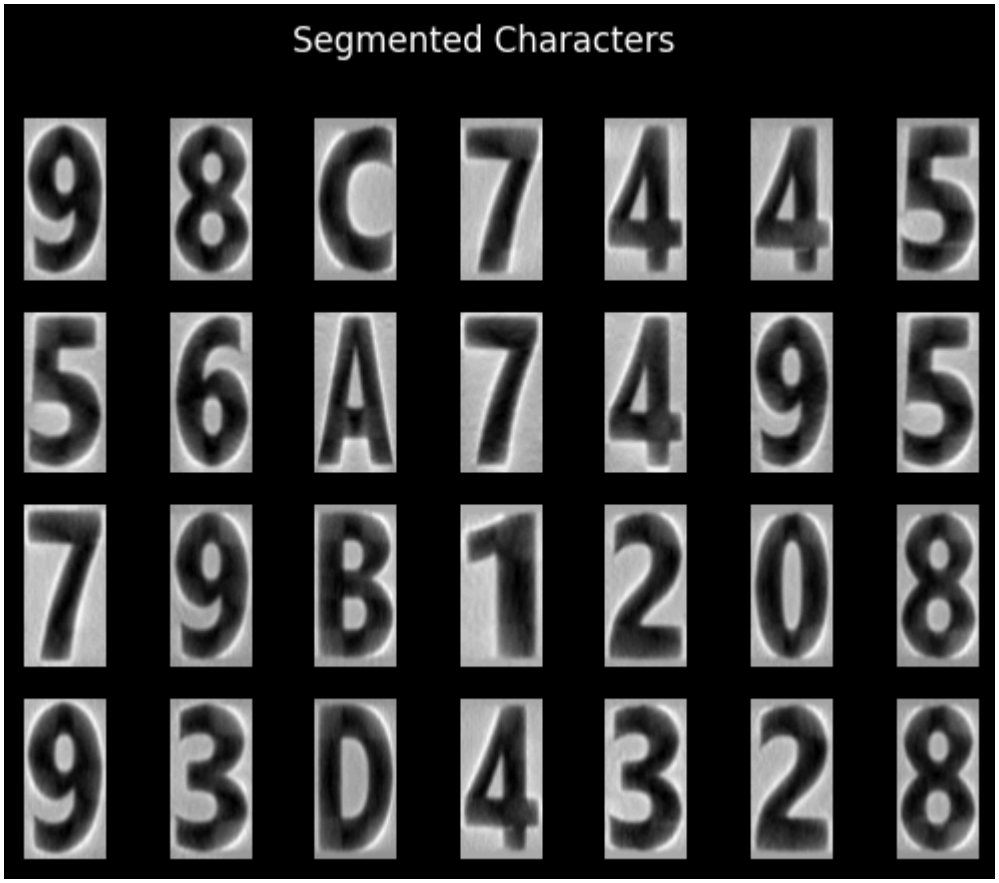
2.2.4. Pass the deblurred image to the character recognition function that you implemented in the ideal section. Observe whether the segmented characters are recognizable using the correlation-based method. If the characters are not correctly identified, or the correlation scores are significantly low, Re-examine your deblurring filter design. Adjust the blur parameter p, the direction of filtering, or the method of inverse filtering. Repeat the deblurring and recognition process until satisfactory results are obtained. (Do this for all the license plates in "realistic" folder)

In [11]:
```
recRealisticCharsList = [recRealImg1, recRealImg2, recRealImg3, recRealIm

for i in range(4):
    recRealisticCharsList[i] = segment_characters(recRealisticCharsList[i

fig3, ax = plt.subplots(4, 7)
for i in range(4):
        for j in range(len(recRealisticCharsList[i])):
            ax[i][j].imshow(recRealisticCharsList[i][j], cmap='gray')
            ax[i][j].axis('off')
fig3.suptitle('Segmented Characters')
```

Out[11]:
```
Text(0.5, 0.98, 'Segmented Characters')
```



In [12]:
```
for i in range(4):
    recognized, corr_vals, best_corrs = recognize_plate(recRealisticChars
    print(recognized)
    print("Best Correlations: ", best_corrs)
```

```
98C7445
Best Correlations:  [0.7741879105549903, 0.7698171015016484, 0.79204763688
97801, 0.7665736582854924, 0.8100867703114017, 0.7939748544393275, 0.79224
94259636604]
56A7495
Best Correlations:  [0.7848640523084403, 0.7616657159432715, 0.80497916933
15813, 0.748966533438102, 0.7886089757084048, 0.7682454470614227, 0.778271
0458918913]
79B1208
Best Correlations:  [0.7827092976466995, 0.765535518381507, 0.765980398106
6926, 0.7943033115622087, 0.7942832935362648, 0.7908110990571771, 0.785076
3670648767]
93D4328
Best Correlations:  [0.7811358922750583, 0.7670252661428992, 0.77519903159
64501, 0.7897504819016125, 0.791777837792712, 0.7874969278615346, 0.790117
8164546382]
```

2.2.5. In this step, analyze the effect of spatial downsampling (i.e., resolution reduction) on character recognition accuracy.

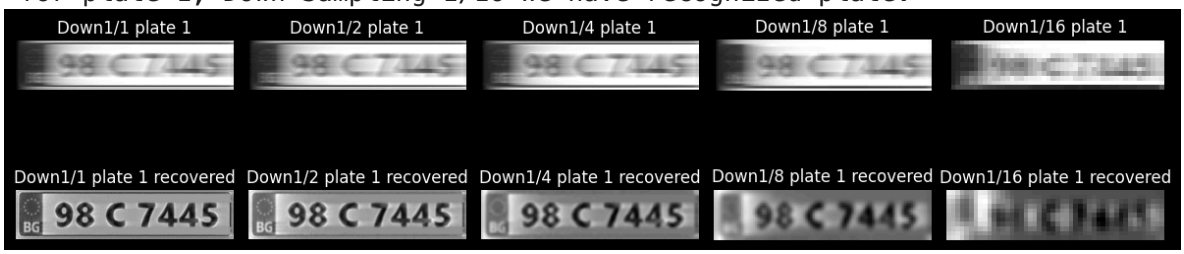
- Apply downsampling to the license plate image before performing character segmentation and also plot before and after.
- Repeat the character segmentation and recognition process for multiple downsampling rates (e.g., 1/2, 1/4, 1/6, ...).
- For each rate, evaluate whether the characters are still recognizable using your correlation-based method.

- Determine and report the minimum sampling rate at which the license plate is still correctly recognizable.
- Plot or tabulate your findings to show how recognition accuracy changes with sampling rate.
- It is not necessary to include all the processes for all the license plates in "realistic" folder in the report. Therefore, include the processing steps for one license plate image in the report, and only report the final outputs for the remaining license plates.

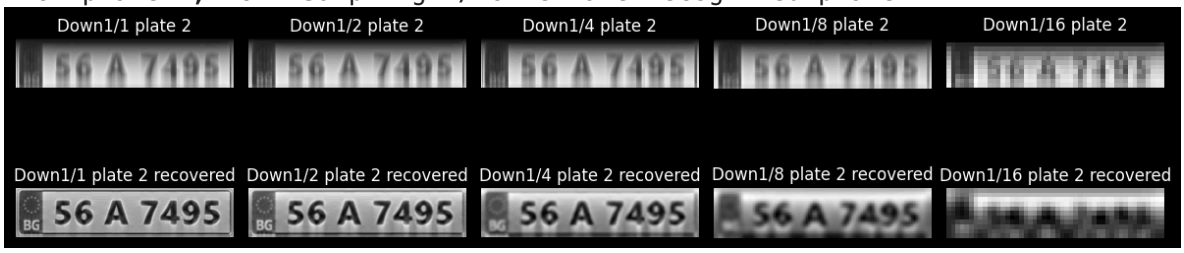
In [13]:
```
realPlates = [realImg1, realImg2, realImg3, realImg4]
side = ["right", "down", "right", "right"]

for j in range(4):
    plt.figure(figsize=(12, 4))
    for i in range(5):
        temp = downsample(realPlates[j], 2**i)
        tempRecover = recover_motion_blurry_img(temp, 0.99, side[j])
        recognized, corr_vals, best_corrs = recognize_plate(segment_chara
        print(f"for plate {j+1}, Down-Sampling 1/{2**i} We have recognize
        plt.subplot(2, 5, i+1), plt.imshow(temp, cmap='gray'), plt.title(
        plt.subplot(2, 5, i+6), plt.imshow(tempRecover, cmap='gray'), plt
    plt.tight_layout()
    plt.show()
```

```
for plate 1, Down-Sampling 1/1 We have recognized plate: 98C7445
for plate 1, Down-Sampling 1/2 We have recognized plate: 98C7445
for plate 1, Down-Sampling 1/4 We have recognized plate: AA
for plate 1, Down-Sampling 1/8 We have recognized plate: 4
for plate 1, Down-Sampling 1/16 We have recognized plate:
```



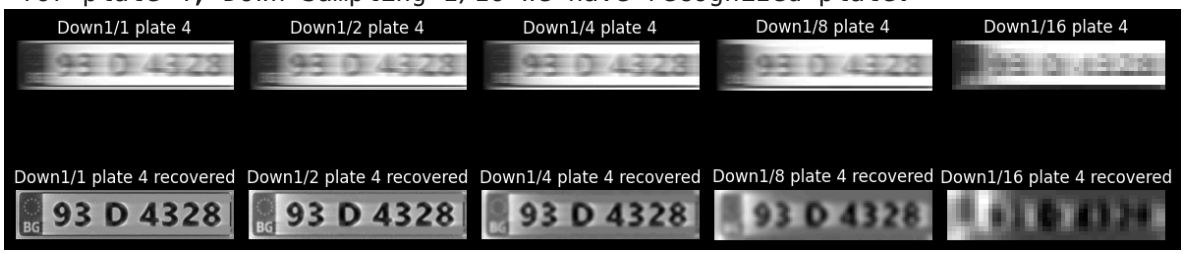

```
for plate 2, Down-Sampling 1/1 We have recognized plate: 56A7495
for plate 2, Down-Sampling 1/2 We have recognized plate: 567495
for plate 2, Down-Sampling 1/4 We have recognized plate:
for plate 2, Down-Sampling 1/8 We have recognized plate:
for plate 2, Down-Sampling 1/16 We have recognized plate:
```




```
for plate 3, Down-Sampling 1/1 We have recognized plate: 79B1208
for plate 3, Down-Sampling 1/2 We have recognized plate: 7981208
for plate 3, Down-Sampling 1/4 We have recognized plate: 7A
for plate 3, Down-Sampling 1/8 We have recognized plate: 1
for plate 3, Down-Sampling 1/16 We have recognized plate:
```




```
for plate 4, Down-Sampling 1/1 We have recognized plate: 93D4328
for plate 4, Down-Sampling 1/2 We have recognized plate: 93D4328
for plate 4, Down-Sampling 1/4 We have recognized plate: AA
for plate 4, Down-Sampling 1/8 We have recognized plate: 4
for plate 4, Down-Sampling 1/16 We have recognized plate:
```



Due to experiments result minimum sampling rate which license plates are still recognizable is 1/2.

2.2.6. In "noisy" folder, there are 2 noisy license plate images. You are asked to determine, without writing any new code (using only existing functions and code), which of these images contains higher-frequency noise content. Provide a convincing justification for your answer.

In [14]:

```
def recover_noisy_motion_blurry_img(img, p=0.99, NSR=0.001):

    img = img.astype(np.float32) / 255.0 # turning unit8 to float32 then

    org_M, org_N = img.shape
    c = org_N
    paddedImg = cv2.copyMakeBorder(img, 0, 0, c, c, cv2.BORDER_REPLICATE)
    M, N = paddedImg.shape

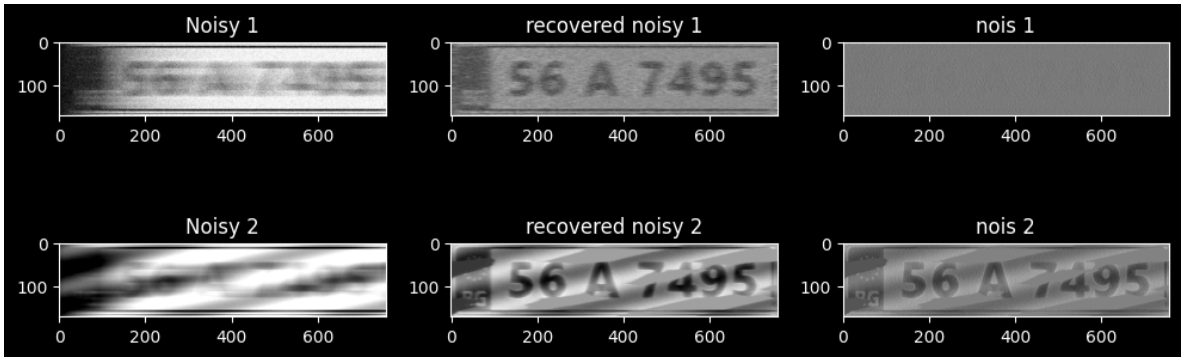
    u = np.arange(N)
    omega_x = 2 * np.pi * u / N
    H_x = (1 - p) / (1 - p * np.exp(-1j * omega_x))

    H = np.tile(H_x, (M, 1))
    G = fftpack.fft2(paddedImg)
    H_conj = np.conj(H)
    F_est = (G * H_conj) / (np.abs(H)**2 + NSR)
    deblurred_image = np.real(fftpack.ifft2(F_est))
    deblurred_image2 = deblurred_image[:, c:c+org_N]
    return deblurred_image2

noisyImg1 = cv2.imread('../noisy/n1.jpg', cv2.IMREAD_GRAYSCALE)
noisyImg2 = cv2.imread('../noisy/n2.jpg', cv2.IMREAD_GRAYSCALE)
recNoisyImg1 = recover_noisy_motion_blurry_img(noisyImg1,0.99,0.01)
recNoisyImg2 = recover_noisy_motion_blurry_img(noisyImg2,0.99,0.001)
nois1 = recover_noisy_motion_blurry_img(noisyImg1,0.99,0)
nois2 = recover_noisy_motion_blurry_img(noisyImg2,0.99,0)


plt.figure(figsize=(12, 4))
```

```
plt.subplot(2, 3, 1), plt.imshow(noisyImg1, cmap='gray'), plt.title('Nois
plt.subplot(2, 3, 2), plt.imshow(recNoisyImg1, cmap='gray'), plt.title('r
plt.subplot(2, 3, 4), plt.imshow(noisyImg2, cmap='gray'), plt.title('Nois
plt.subplot(2, 3, 5), plt.imshow(recNoisyImg2, cmap='gray'), plt.title('r
plt.subplot(2, 3, 3), plt.imshow(nois1, cmap='gray'), plt.title('nois 1')
plt.subplot(2, 3, 6), plt.imshow(nois2, cmap='gray'), plt.title('nois 2')
plt.show()
```



To achieve a clearly recognizable license plate image, we used a Noise-to-Signal Ratio (NSR) of 0.01 for the first image and 0.001 for the second. The higher quality of the second image suggests that the first image contained more noise.

Further confirmation comes from applying a high-frequency filter with NSR = 0 to both images. The first image retained significantly more noise, indicating the presence of higher-frequency noise components compared to the second.
